

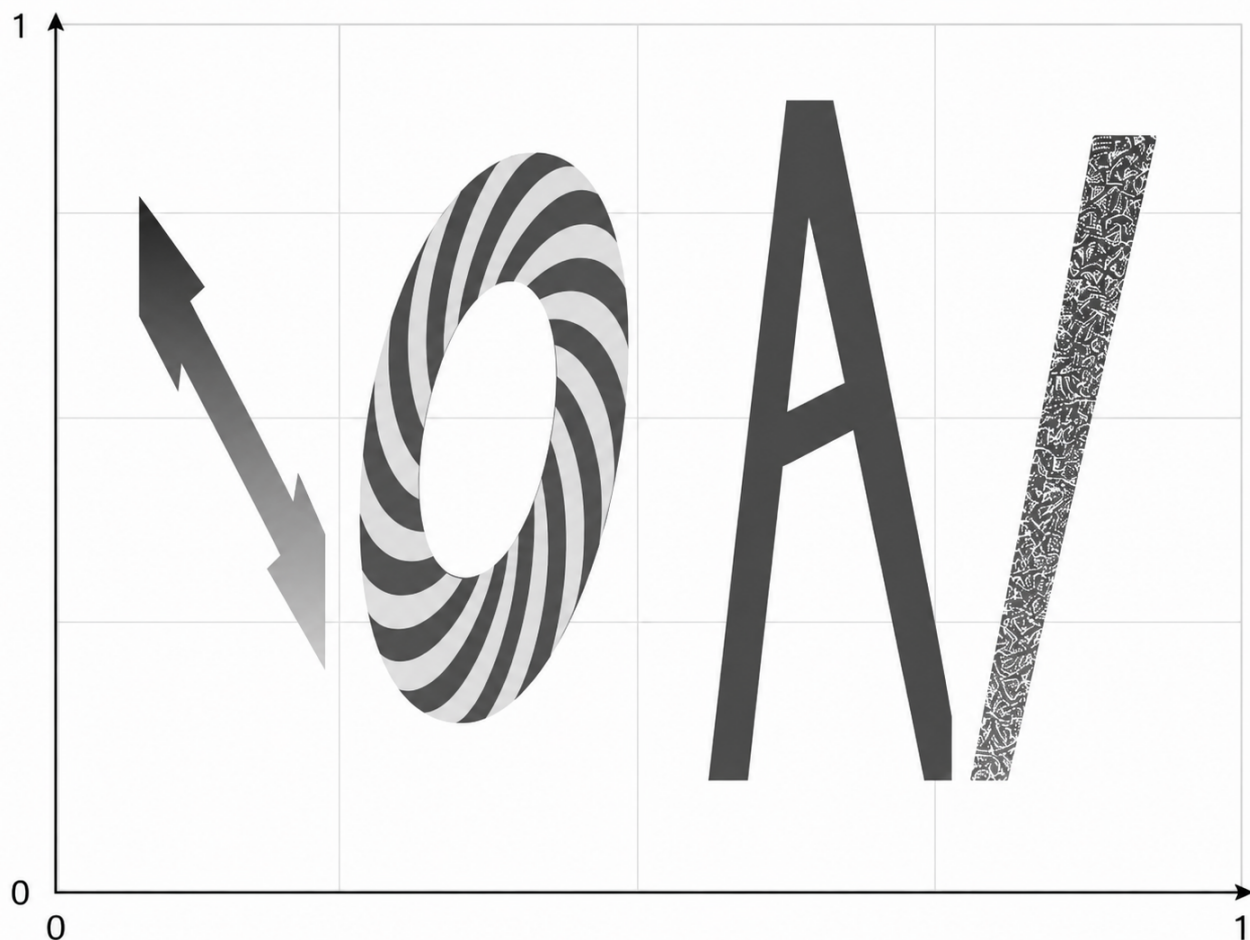
Pole IOAI

- **Časový limit:** 5 minut
- **Úložiště:** 5 GB
- **Velikost řešení:** `solution.ipynb`, `custom_model.py` dohromady ≤ 1 MB
- **Předtrénované modely:** žádné — trénujte od začátku, během hodnocení není k dispozici internet
- **Výchozí skóre:** 31.2187

Úloha

Starosta Astany chce město vyzdobit stylizovanými logy IOAI. Jako statistik vnímá vše — včetně loga — jako prostorovou funkci $F(x, y, \overline{W})$, kde $x, y \in [0, 1]$ představují souřadnice ve 2D rovině a $\overline{W}$ je množina skrytých parametrů určujících stylistické vlastnosti, jako jsou barvy a úhly písmen.

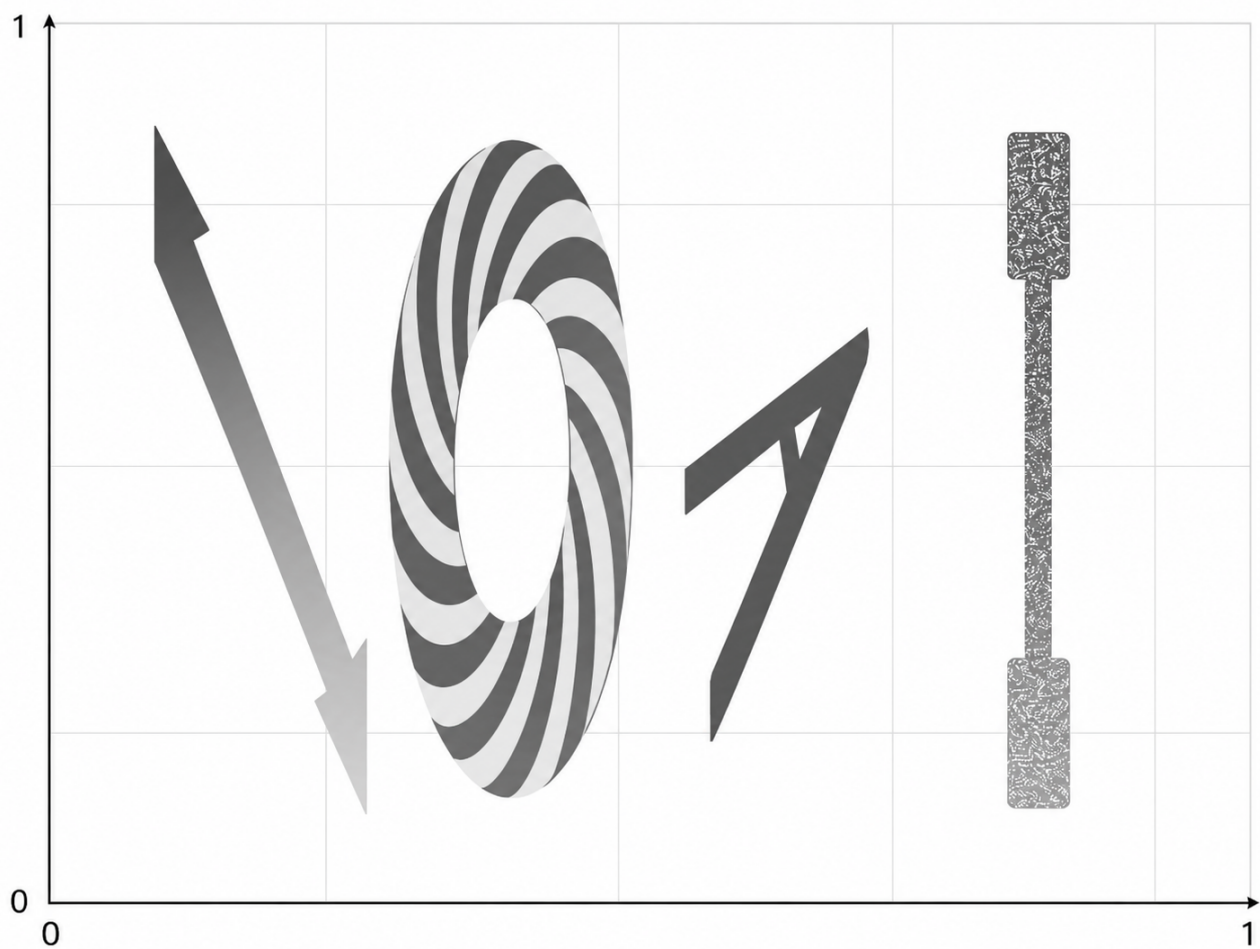
Protože F je příliš složitá na to, aby ji bylo možné vyjádřit explicitní matematickou rovnicí, je vaším úkolem natrénovat neuronovou síť, která ji bude aproximovat. Síť bude pro libovolnou dvojici souřadnic (x, y) vracet hodnotu **pole IOAI**, a vytvářet tak úplnou vizualizaci loga v podobě tepelné mapy napříč rovinou. Zde je příklad vizualizace F v podobě tepelné mapy pro určité konkrétní skryté parametry $\overline{W}$.

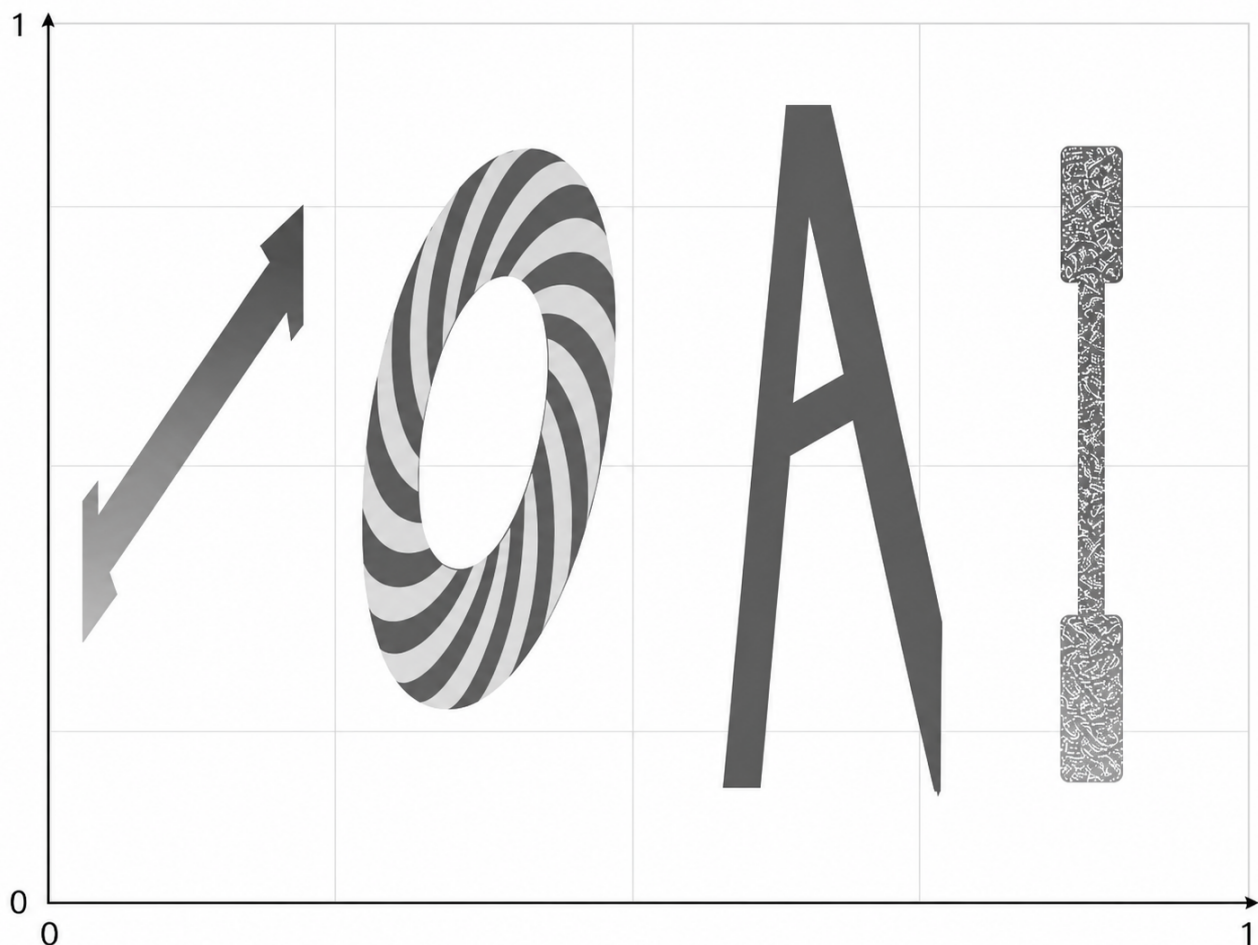


Z čeho se pole IOAI skládá? Ze čtyř písmen a pozadí.

- Hodnoty uvnitř prvního písmene $\mathbb{I}$ jsou velmi vysoké ($1e+10$ a více) a mají lineární gradient
- Hodnoty v písmenu $\circ$ vykazují spirálový vzor
- Hodnota uvnitř písmene A je vždy -1
- Hodnoty uvnitř posledního písmene $\mathbb{I}$ mají být náhodnými hodnotami z rozsahu $[-2026, 2026]$, i když je tentýž bod vyhodnocen dvakrát
- Mimo písmena je hodnota vždy nulová

Funkce má skryté parametry $\overline{W}$, které ovlivňují měřítko a sklon písmen spolu s rozsahem hodnot uvnitř prvního písmene $\mathbb{I}$. Písmena se však nebudou protínat. Zde je několik ilustrativních příkladů toho, jak pole IOAI vypadá s různými $\overline{W}$:





Co máte k dispozici:

Tato úloha neobsahuje ŽÁDNÉ datasety. Místo toho máte k dispozici generující funkci, která je nakonfigurována souborem JSON s konfigurací v umístění `data/train_config/field_config.json`.

Testovací konfigurace je skrytá, ale je obdobného charakteru. Vaším úkolem je natrénovat model na poskytnutém generátoru s použitím libovolného množství dat. Vaše „trénovací“ a „testovací“ rozdělení jsou generována týmhž generátorem — pouze nevíte, ve kterých bodech (x_i, y_i) budete hodnoceni.

Vaše odevzdání se má skládat z:

- třídy trénovaného modelu uložené jako `custom_model.py`. Tento model má dědit od třídy `torch.nn.Module` a používat pouze `importy torch`. Má obsahovat třídu `CustomModel` použitou v notebooku `solution.ipynb`.
- notebooku `solution.ipynb`, který vytvoří váhy `model.pt`

Hodnocení

Pro každou oblast je minimální skóre 0 a maximální skóre 1. Konečné skóre je průměrem skóre ze všech pěti oblastí (čtyři pro jednotlivá písmena a jedna pro pozadí) vynásobeným 100. Uplatňuje se **penalizace za počet parametrů**:

Pokud má váš model více než 20260 parametrů, skóre se sníží na polovinu.

Počet parametrů se měří pomocí `sum(p.numel() for p in model.parameters())`. Očekáváme, že váš model bude fungovat také ve stochastickém režimu, přičemž součástí modelu bude prvek PyTorch `nn.Dropout`.

Pro standardní oblasti

Pro každou oblast R (první písmeno `I`, `O`, `A`, `Background`) vyhodnocujeme model na $N_R = 512$ testovacích bodech (x_i, y_i) se skutečnými hodnotami v_i a predikcemi $\hat{v}_i$. Jako hlavní metriku používáme normalizovanou střední absolutní chybu (MAE). MAE je definována jako:

$$\text{MAE}(R) = \frac{1}{N_R} \sum_{i=1}^{N_R} |\hat{v}_i - v_i|.$$

A normalizace se provádí jako

$$\text{score}(R) = 1 - \min \left(\frac{\text{MAE}(R)}{s_R}, 1 \right),$$

kde $s_R > 0$ je konstanta měřítka.

Pro oblast posledního písmene `I`

V této oblasti je **během vyhodnocování povolen dropout**. Pro každý testovací bod j :

1. Spustíme model $K = 10$ krát, abychom získali $\hat{v}_{j,1}, \dots, \hat{v}_{j,K}$.
2. Pokud je kterýkoli výstup mimo rozsah $[-2026, 2026]$, pak $\text{pointScore}(j) = 0$.
3. Jinak vypočítáme směrodatnou odchylku σ_j z K výstupů a převedeme ji na skóre:

$$\text{pointScore}(j) = \min \left(\frac{\sigma_j}{s_E}, 1 \right),$$

kde $s_E > 0$ je pevná konstanta měřítka.

Skóre oblasti je průměrem přes všechny body v dané oblasti:

$$\text{score}(I_{last}) = \frac{1}{N_E} \sum_{j=1}^{N_E} \text{pointScore}(j),$$

kde $N_E = K * N_R$.

Jednoduše řečeno, čím větší rozmanitosti dosáhnete, tím vyšší bude vaše skóre pro tuto oblast. **Náhodnost nemůžete používat v čisté podobě, včetně funkcí PyTorch `rand*` a `_uniform`; náhodnost musí pocházet z inference se zapnutým dropoutem.**

Jak odevzdat řešení

1. Otevřete `solution.ipynb` a spustte všechny buňky.
2. Vylepšete model `CustomModel` v `custom_model.py`
3. Ujistěte se, že vaše poslední buňka uloží model do souboru `model.pt`.
4. Na kartě Git v JupyterLab zařadte do stage, okomentujte a commitněte `solution.ipynb` a `custom_model.py` a poté je pushněte.
5. Vratte se na stránku soutěže a klikněte na **Odevzdat**. Komentář k odevzdání má být stejný jako komentář z předchozího kroku.